CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

PROJECT REPORT

ON

STUDENT MARKS AND GRADE MANAGEMENT SYSTEM

Using Arrays and Structures in C Programming

Program Name: BCA – Data Science
Subject: Computer Programming

Submitted to:

Kamaldeep Mam

Submitted by:

Name: Siddharth

UID: 26BCD10283

Section: BCD-5A

ACADEMIC PROJECT

STUDENT MARKS AND GRADE MANAGEMENT SYSTEM

1. Introduction

A Student Marks and Grade Management System is a simple computer-based application developed using the C programming language. Its purpose is to store student information, calculate total and average marks, and assign grades automatically. This project demonstrates arrays and structures: an array stores subject marks, while a structure groups a student's name, roll number, marks, total, average and grade. The system reduces manual calculations and makes basic result management faster and more organized.

2. Objectives of the Project

- Store the names and roll numbers of multiple students.
- Enter marks for five subjects for each student.
- Calculate each student's total and average marks.
- Assign a grade according to the calculated average.
- Display complete student result details.
- Understand the practical use of arrays and structures in C.

3. Problem Statement

Schools and colleges need to maintain student records and calculate examination results. Performing these calculations manually for many students can take time and may lead to mistakes. This project provides a simple C program that accepts student details and marks, calculates totals and averages, and assigns grades automatically.

4. Concepts Used in the Project

4.1 Arrays

An array is a collection of elements of the same data type stored under one variable name. In this project, an integer array stores the marks of five subjects. Example: int marks[5];

4.2 Structures

A structure is a user-defined data type in C that groups different types of information under one name. The Student structure stores the student's name, roll number, subject marks, total, average and grade.

4.3 Loops

The for loop repeats instructions. It is used to accept marks for each subject and to display the records of all students.

4.4 Conditional Statements

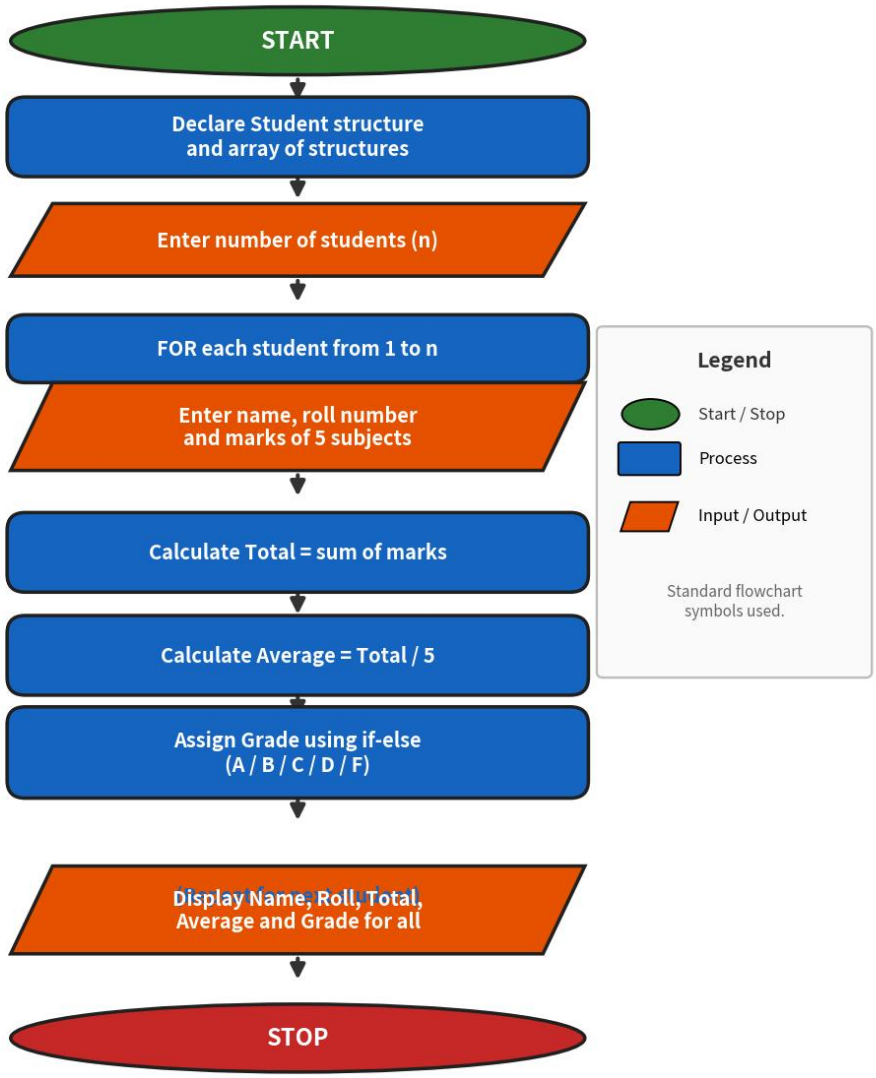
The if-else statements compare the average marks with grade boundaries and assign the appropriate grade.

5. Algorithm

1. Start the program.
2. Declare a structure named Student.
3. Enter the number of students.
4. For each student, enter the name and roll number.
5. Enter marks for five subjects.
6. Calculate the total marks.
7. Calculate the average by dividing the total by five.
8. Use conditional statements to assign a grade.
9. Display the student details, total, average and grade.
10. Stop the program.

6. Flowchart

The flowchart uses standard symbols: oval for Start/Stop, rectangle for Process, and parallelogram for Input/Output. The input, calculation and grade steps are repeated for every student.



Note: The input–calculate–grade steps are repeated for each of the n students.

7. Complete C Program

The following program accepts student details and five subject marks, calculates totals and averages, assigns grades, and displays each result.

```
#include <stdio.h>

struct Student {
    char name[50];
    int roll;
    int marks[5];
    int total;
    float average;
    char grade;
};

int main() {
    struct Student s[50];
    int n, i, j;

    printf("Enter number of students: ");
    scanf("%d", &n);

    if (n < 1 || n > 50) {
        printf("Please enter between 1 and 50 students.");
        return 1;
    }

    for (i = 0; i < n; i++) {
        printf("\\nEnter details of student %d\\n", i + 1);
        printf("Enter name: ");
        scanf(" %49[^\\n]", s[i].name);
        printf("Enter roll number: ");
        scanf("%d", &s[i].roll);
        s[i].total = 0;

        for (j = 0; j < 5; j++) {
            printf("Enter marks of subject %d: ", j + 1);
            scanf("%d", &s[i].marks[j]);
            s[i].total += s[i].marks[j];
        }
        s[i].average = s[i].total / 5.0;

        if (s[i].average >= 90)      s[i].grade = 'A';
        else if (s[i].average >= 80) s[i].grade = 'B';
        else if (s[i].average >= 70) s[i].grade = 'C';
        else if (s[i].average >= 60) s[i].grade = 'D';
        else                         s[i].grade = 'F';
    }

    printf("\\n----- STUDENT RESULT -----\\n");
    for (i = 0; i < n; i++) {
        printf("\\nName: %s", s[i].name);
        printf("\\nRoll Number: %d", s[i].roll);
        printf("\\nTotal Marks: %d", s[i].total);
        printf("\\nAverage Marks: %.2f", s[i].average);
        printf("\\nGrade: %c\\n", s[i].grade);
    }
    return 0;
}
```

8. Explanation of the Program

Step 1: Header file
#include <stdio.h> provides standard input and output functions such as printf() and scanf().

Step 2: Structure declaration
The Student structure groups the name, roll number, marks, total, average and grade.

Step 3: Array of structures
struct Student s[50]; creates space for records of up to 50 students.

Step 4: Taking input
The program accepts the student's name, roll number and marks in five subjects.

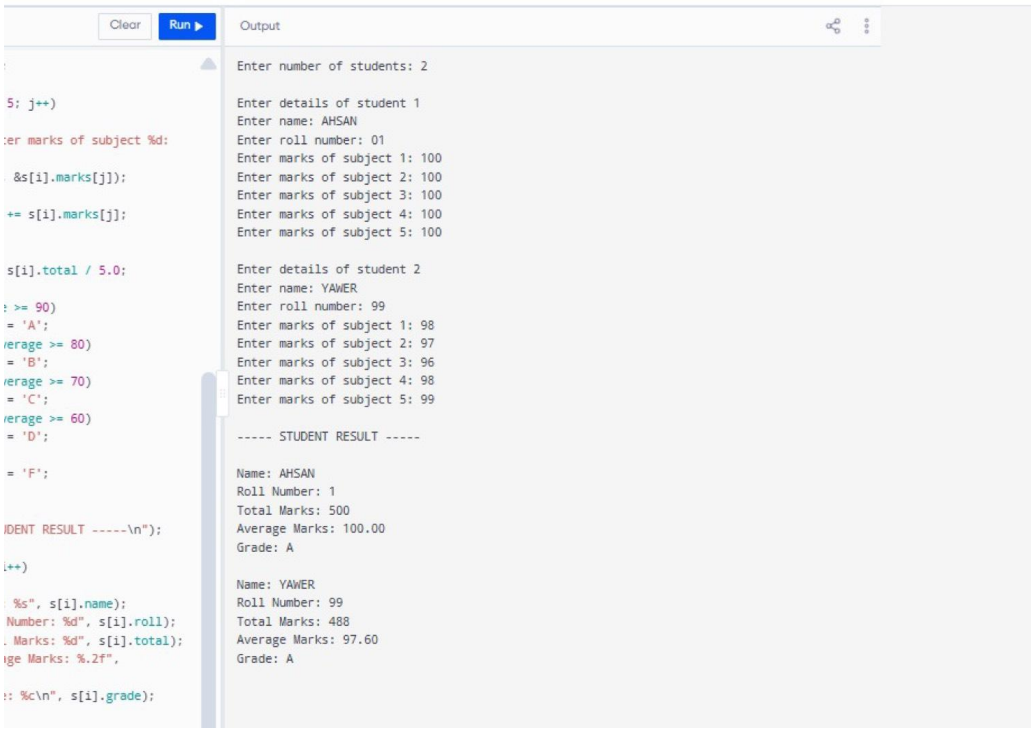
Step 5: Calculating total
A loop adds the five subject marks and stores the result in the total variable.

Step 6: Calculating average
The total is divided by 5.0 to calculate the average as a floating-point value.

Step 7: Grade calculation
The if-else ladder checks the average and assigns grade A, B, C, D or F.

Step 8: Displaying results
A final loop displays each student's name, roll number, total marks, average and grade.

9. Sample Output
The sample execution below shows the program output for two students. Student 1 (AHSAN) scored full marks and received Grade A. Student 2 (YAWER) also performed excellently and received Grade A.



10. Grading System

Average Marks	Grade
90–100	A
80–89.99	B
70–79.99	C
60–69.99	D
Below 60	F

These grade boundaries are illustrative. They can be changed according to the grading policy followed by an institution.

11. Advantages of the Project

- Saves time by calculating marks automatically.
- Reduces errors in manual calculations.
- Stores multiple student records in one program.
- Makes student information easier to organise.
- Automatically assigns grades.
- Demonstrates useful C programming concepts.

12. Limitations of the Project

- The program can store up to 50 students.
- Student records are not permanently saved in a file.
- The program uses a fixed number of five subjects.
- It does not provide options to edit or delete records.
- It does not include a graphical user interface.

13. Applications of the Project

- Maintaining basic student result records in schools and colleges.
- Supporting examination and academic record keeping.
- Demonstrating the foundation of a student database application.
- Practising arrays, structures, loops and conditional statements.

14. Future Scope

- Save student records permanently using file handling.
- Search students by roll number.
- Add options to update and delete student records.
- Support a variable number of subjects.
- Generate printable result reports.
- Develop a graphical user interface.

15. Conclusion

The Student Marks and Grade Management System is a useful beginner-level project developed using the C programming language. It demonstrates how arrays and structures can be used together to store and manage student information. The program calculates total marks, average marks and grades using loops and conditional statements. It helps reduce manual effort and improves understanding of basic programming concepts. Overall, the project shows how C programming can be applied to a simple real-life problem.

16. References

- E. Balagurusamy, Programming in ANSI C.
- Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language.
- C programming classroom notes and practical exercises.
- C language reference documentation.